

Apunte 46 — Por qué un programa tarda en abrir: importar tarde y no calcular dos veces

Proyecto Froth · aprender Machine Learning construyendo

1 Glosario en palabras simples

- **Importar:** cuando tu programa dice `import umap`, Python va al disco, busca esa librería, la lee entera y la deja lista en memoria. Eso tarda. Y ocurre *aunque nunca llegues a usarla*.
- **Librería:** código que escribió otra gente y tú reutilizas. UMAP, HDBSCAN y PyTorch son librerías.
- **Nivel de módulo:** las líneas que están pegadas al margen izquierdo de un archivo `.py`. Se ejecutan en cuanto alguien importa ese archivo, sin que nadie llame a nada.
- **Import perezoso** (*lazy import*): mover el `import` DENTRO de la función que lo necesita. Así el coste se paga la primera vez que usas esa función, no al abrir el programa.
- **Caché:** guardar un resultado ya calculado para no repetirlo. La palabra se pronuncia “cash”.
- **Caché en memoria RAM:** se guarda mientras el programa está vivo. Si cierras el programa, se pierde.
- **Caché en disco:** se guarda en un archivo. Sobrevive a cerrar el programa, a apagar el ordenador y a volver mañana.
- **Clave de caché:** la “etiqueta” con la que guardas el resultado. Si la etiqueta cambia, el resultado guardado ya no vale y hay que recalcular.

2 El problema, tal como apareció

Batman escribió: “*el tiempo de espera es mucho*”. Frase corta, diagnóstico cero. La regla de esta casa es no adivinar: se mide.

Se cronometró cada pieza del arranque de Froth por separado. Esto salió:

Pieza del arranque	Segundos
importar <code>umap</code> + <code>hdbscan</code>	24,8
importar <code>torch</code> + <code>sentence_transformers</code>	19,7
importar <code>streamlit</code>	2,1
cargar el modelo SPECTER	2,5
barrido DBCV (elegir la granularidad)	2,3
leer los datos del disco	0,2
TOTAL	52,2

Concepto: Lo que enseña la tabla

44,5 de los 52,2 segundos son *importar librerías*. Y la pantalla de inicio de Froth (el título, el desplegable de temas, la caja para pegar un título nuevo) **no usa ninguna de ellas**. Estabas esperando 45 segundos por código que no se iba a ejecutar.

3 Primer arreglo: importar tarde

Antes, arriba del todo de `froth/cluster.py`:

```
import umap
import hdbscan
```

Después, dentro de la función que de verdad lo usa:

```
def reduce_to_2d(vectors):
    import umap          # se paga aquí, no al abrir la app
    reducer = umap.UMAP(n_components=2, metric="cosine", random_state=42)
    return reducer.fit_transform(vectors)
```

Ojo / error típico

El coste no desaparece: se mueve. UMAP sigue tardando sus 30 segundos. La diferencia es *cuándo*. Antes lo pagabas mirando una pantalla en blanco, sin saber si el programa estaba vivo. Ahora lo pagas cuando cosechas un tema nuevo, con una barra de progreso delante que te dice qué está haciendo. La misma espera se siente completamente distinta según si sabes por qué esperas.

4 Pero faltaba lo gordo, y estaba escondido

Con los imports arreglados, el arranque medido en frío bajó a 8,8 segundos. Y sin embargo, al abrir la app de verdad, **seguía tardando más de tres minutos y medio**.

La lección está justo aquí: **el cronómetro solo mide lo que le pides que mida**. Yo había medido los imports, no la app entera. Al mirar lo que la app imprimía mientras trabajaba apareció el culpable de verdad:

Naming the subtopics (reading abstracts)...

Poner nombre a los 44 subtemas. Para cada uno hay que leer sus abstracts, sacar frases candidatas, convertirlas en vectores con SPECTER y quedarse con la que mejor representa al grupo (es el apunte 40). Eso es caro. Y se hacía **entero, cada vez que abrías la aplicación**.

Concepto: Por qué se repetía si ya estaba “cacheado”

El código ya usaba `@st.cache_data`, que guarda el resultado **en memoria RAM**. Funciona de maravilla mientras la app está abierta: cambias de pestaña y no recalcula. Pero al cerrar la app, esa memoria se libera. Al día siguiente vuelves a pagar los tres minutos. Una caché en RAM te protege de repetir dentro de una sesión. No te protege de repetir *entre* sesiones. Para eso hace falta el disco.

5 Segundo arreglo: guardar en disco, con la clave correcta

Los nombres se guardan ahora en un archivo `.json`. La parte interesante no es guardar: es **con qué etiqueta**.

```
key = f"{version}|{mcs}|{years[0]}-{years[1]}"
digest = hashlib.sha1(key.encode()).hexdigest()[:12]
# -> 2_Datos/processed/subtitles_36e4d383cfce.json
```

La etiqueta mezcla las tres cosas de las que dependen los nombres:

- **la versión de los datos:** si cosechas más papers, los subtemas cambian;
- **la granularidad:** mover el deslizador crea otros subtemas;
- **el rango de años:** filtrar por años deja fuera papers, y los grupos cambian.

Si mueves cualquiera de las tres, la etiqueta cambia, el archivo guardado no aparece, y se recalcula. Si no mueves ninguna, se lee del disco al instante.

Ojo / error típico

Este es el error clásico de las cachés, y da miedo porque no falla ruidosamente. Si la clave se te queda corta (por ejemplo, solo el nombre del corpus), el programa te enseña los nombres viejos sobre datos nuevos. No se rompe, no avisa: simplemente te miente. Una caché con la clave mal puesta es peor que no tener caché.

Concepto: Esto NO contradice la regla 9

Podría parecer que guardar el resultado es lo contrario de “recalcular para cada corpus”. No lo es. Se sigue calculando para *este* corpus, con *sus* datos. Lo único que se elimina es calcular **dos veces la misma pregunta**. Un valor por defecto sería no medir nunca; esto es medir una vez y acordarse.

6 El resultado

	Antes	Ahora
Abrir la app hasta poder usarla	>3,5 min	≈30 s
Solo cargar librerías	52,2 s	8,8 s
¿Carga SPECTER al abrir?	sí	no

Pruébalo tú (teclado en mano)

Compruébalo tú, y de paso aprende a medir:

1. Abre Froth con el acceso directo del escritorio y cuenta mentalmente hasta que veas la pantalla de inicio.
2. Ahora mueve el deslizador de granularidad a un número que no hayas usado nunca (por ejemplo 7). Fíjate: *ahí* sí espera, porque esa combinación no está guardada todavía.
3. Vuelve a moverlo a 18 y luego otra vez a 7. La segunda vez es instantánea: ya está en el disco.
4. Mira la carpeta `2_Datos\processed\`. Verás un archivo `subtitles_...json` por cada combinación que hayas probado. Ábrelo con el Bloc de notas: son los nombres de tus subtemas.

En una frase

Si un programa tarda en abrir, mide antes de tocar: casi siempre está haciendo trabajo que no hace falta todavía (impórtalo tarde) o trabajo que ya hizo ayer (guárdalo en disco, con una clave que incluya TODO lo que puede cambiar el resultado).